Why run_sql Has a Factory

Why the SQL tool is built by `make_run_sql(gateway)` instead of being a plain `@tool` function. See also the architecture map.

The problem

When the LLM wants data, it can only say one thing: *"run this SQL"*. To run it, the tool needs two things:

WHAT	WHO PROVIDES IT	WHEN
`sql`	The LLM, as text	On every tool call
`gateway`	Our code: the gateway around BigQuery, with your project and limits from `.env`	Once, at startup

The LLM can't supply the gateway. So the tool must already have it before the LLM ever calls it.

The code

```
# tools.py
def make_run_sql(gateway: QueryGateway) -> BaseTool:   # the factory: receives the gateway once
    @tool
    def run_sql(sql: str) -> str:                      # the tool: the LLM only fills in "sql"
        """Run one SELECT statement against the retail database and return the rows as a table."""
        try:
            result = gateway.run(sql)                  # gateway is remembered from the outer functi
        except QueryError as error:
            return f"Error: {error}"                   # rejected SQL goes back to the LLM to fix
        return format_result(result)                   # table + "cut off" / "no rows" notes

    return run_sql                                     # hands back the finished tool

# app.py, assemble()
agent = build_agent(model, [make_run_sql(gateway)], system_prompt, settings.max_llm_calls, checkpoir
#                           └── called here: the list holds the tool, not the factory
```

`@tool` turns a function's parameters into what the LLM must fill in. That's why it decorates the inner `run_sql(sql)`. On `make_run_sql(gateway)` it would ask the LLM for a gateway, which it can't give. The inner function remembering `gateway` from the outer one is a **closure**.

Why not a plain tool?

Gateway built at import

```
gateway = QueryGateway(BigQueryDatabase(...)

@tool
def run_sql(sql: str) -> str:
    return format_result(gateway.run(sql))
```

- Runs when Python imports `tools.py`, before `.env` is loaded
- Importing it connects to real BigQuery, so tests need keys and network
- Settings get read inside `tools.py` instead of one place

Global set later

```
gateway = None

@tool
def run_sql(sql: str) -> str:
    return format_result(gateway.run(sql))

# somewhere else, before first use:
tools.gateway = QueryGateway(...)
```

- Forget to set it and the first question crashes
- All tests share one global
- To understand the tool you must find who sets it

What the factory gives us

1. We control the order, not Python's imports

Importing `tools.py` builds nothing. `app.py` builds everything explicitly, in the order we choose:

1. Read `.env` into `Settings`
2. Connect to BigQuery: `BigQueryDatabase(...)`
3. Build the guard from the schema's tables: `QueryGuard(dialect, allowed_tables)`
4. Wrap the database with the rules: `QueryGateway(database, guard, max_rows, max_bytes, scope)`
5. Give the tool its gateway: `make_run_sql(gateway)`
6. Give the agent its tool: `build_agent(model, [run_sql], prompt, max_llm_calls, checkpointer)`

2. The caller chooses what the tool is connected to

```
# real run
run_sql = make_run_sql(QueryGateway(BigQueryDatabase(project, max_bytes), guard, max_rows=50, ma

# later steps: a stricter gateway (PII rules), same tool code
run_sql = make_run_sql(stricter_gateway)
```

3. Testability

Tests pass a gateway built on a fake database, through the same wiring. No BigQuery, no keys, no network; same result every time.

```
# tests/test_turn.py
database = FakeDatabase([QueryResult(columns=["brand", "revenue"], rows=[("Brand A", 100.0)])])
model    = ScriptedChatModel(script=[run_sql_call(sql), AIMessage("Brand A leads with $100.")])

result = assemble_app(model, database).ask("Which brand has the most revenue?")

assert database.executed == [sql]      # the tool really went through the gateway to the fake
assert result.answer == "Brand A leads with $100."
```

In one line

> The factory lets the caller decide what the tool is connected to (real BigQuery, a stricter gateway, or a fake) instead of the tool deciding for itself at import time. The general idea is called **dependency injection**: code is given what it needs instead of creating or finding it itself.